

PHISHSHIELD AI

Tech Stack & Team Learning Guide

Everything the team needs to learn to build, explain and defend the project — taken from the *current repository*, not from the idea note.

Running product name in code: **CyberGuard API 2.0**

SIH product name: **PhishShield AI**

Stack in one line: React 19 + Vite 8 + FastAPI + SQLite + Manifest V3 + weighted fusion (no trained CNN)

Generated from repository inspection · 17 August 2026

Honesty rule for every viva answer: FastAPI not Flask. SQLite not live PostgreSQL. Weighted fusion, not a trained CNN. Confidence is signal agreement, not accuracy. The extension does not read native SMS.

Subtitle: Everything the team needs to learn to build, explain and defend the project

Product names in this repo: code and API title use **CyberGuard**; SIH materials use **PhishShield AI**. Say both in the viva: "PhishShield AI is the product name; the running API is CyberGuard 2.0."

Source of truth: this document is based on the **current codebase**, not the original idea note. If README wording and code disagree, **code wins**.

Inspection date: 17 August 2026

Table of contents

1. How this guide was built
2. Exact tech stack (inventory)
3. Technology categories
4. Explain every major technology
5. Team learning roadmap
6. Role-based learning plan
7. Architecture (as implemented)
8. Trace one complete request
9. AI / ML (honest)
10. Psychological threat detection
11. Multi-channel analysis
12. Risk score
13. Security
14. Database (SQLite, not live PostgreSQL)
15. API catalogue
16. Browser extension
17. Jury questions
18. Explain this to the jury
19. Technology difficulty
20. Five-day crash course
21. One-page cheat sheet
22. Code vs documentation discrepancies

1. How this guide was built

The repository was inspected: `frontend/package.json`, `backend/requirements.txt`, `backend/app/main.py`, all routers under `backend/app/routes/`, services under `backend/app/services/`, `backend/app/database/connection.py`, `backend/app/models/`, `frontend/src/`, `extension/`, `backend/tests/`, `README.md`, `frontend/.env.example`, `backend/app/config.py`.

Not found in this repo (do not claim them as implemented): Flask, live PostgreSQL, Alembic, Docker, scikit-learn, TensorFlow, Keras, a trained `.pkl` / `.h5` model, spaCy, BERT, SlowAPI rate limiting, Gmail/Outlook OAuth, native Android SMS.

2. Exact tech stack

STATUS values:

- **Implemented** — used by live code paths
- **Partial** — optional, incomplete, or weakly configured
- **Planned** — mentioned or stubbed, not a working product feature
- **Not implemented** — absent
- **Dependency present, usage not confirmed** — listed in requirements or models, not used by the live request path

Domain	Technology	Actual use	Files / location	Status	Learning priority
Frontend	React 19.2.8	Dashboard UI: Scanner, History, Reports, Statistics, auth pages	frontend/src/App.jsx, pages/, components/	Implemented	HIGH
Frontend	React Router DOM 7	Client-side routes and ProtectedRoute for dashboard pages	frontend/src/App.jsx, components/ProtectedRoute.jsx	Implemented	HIGH
Frontend	Axios	HTTP client to FastAPI; attaches JWT from localStorage	frontend/src/services/api.js	Implemented	HIGH
Frontend	Lucide React	Icons in layouts and pages	frontend/src/layouts/, pages/	Implemented	LOW
Frontend	Recharts	Charts on Statistics / Dashboard	frontend/src/pages/Statistics.jsx, Dashboard.jsx	Implemented	MEDIUM
Frontend	Vite 8	Dev server on 127.0.0.1:5173 and production bundler	frontend/vite.config.js, package.json	Implemented	MEDIUM
Frontend	Tailwind CSS 4 (@tailwindcss/vite)	Utility CSS styling	frontend/src (className), vite.config.js	Implemented	MEDIUM
Frontend	eslint	JS/JSX lint (npm run lint)	frontend/.eslintrc.json	Implemented	LOW
Backend	Python 3	Language of FastAPI services and tests	backend/app/**/*.*.py, backend/tests/	Implemented	HIGH
Backend	FastAPI	REST API: analyze, threat, user, stats, scans, reports	backend/app/main.py, routes/	Implemented	HIGH
Backend	Uvicorn	ASGI server, host 0.0.0.0 port 8000	backend/app/main.py, README.md	Implemented	HIGH
Backend	Pydantic v2	Request body validation (ContentRequest, UserRegister, etc.)	backend/app/routes/*.py	Implemented	HIGH
Backend	pydantic-settings	Settings object (SECRET_KEY, DATABASE_URL string)	backend/app/config.py	Partial	MEDIUM
Backend	python-dotenv	Load backend/.env (SECRET_KEY, GSB key)	backend/app/services/auth.py, external_intel.py	Implemented	HIGH
Backend	requests	Outbound HTTP: Safe Browsing, OpenPhish, RDAP, redirects	backend/app/services/external_intel.py, redirects.py, ad_signals.py	Implemented	HIGH
Backend	httpx	Listed for tests / HTTP client	backend/requirements.txt, tests/	Implemented	MEDIUM
Backend	python-multipart	QR/screenshot file uploads	backend/app/routes/analyze.py	Implemented	MEDIUM
Backend	SQLAlchemy 2	ORM model files exist (User, UrlCheck, Threat)	backend/app/models/, requirements.txt	Dependency present, usage not confirmed in live routes	LOW

Domain	Technology	Actual use	Files / location	Status	Learning priority
Database	SQLite (sqlite3)	Live persistence: users, url_checks, content_scans, reports	backend/app/database/connection.py, backend/threats.db	Implemented	HIGH
Database	PostgreSQL	Not connected. config.py default is sqlite URL, not Postgres	backend/app/config.py comment only	Planned	MEDIUM
Database	Alembic	No alembic.ini or migrations folder	not in repo	Not implemented	LOW
AI / Fusion	Weighted signal fusion (not a trained model)	Combines url/domain/ml/nlp/psych/intel/brand/redirect/sender/community	backend/app/services/risk_engine.py	Implemented	HIGH
AI / Fusion	Rule-based URL checker	Heuristics: TLD, brand typos, shorteners, keywords, trusted list	backend/app/services/url_checker.py	Implemented	HIGH
AI / Fusion	scikit-learn / TensorFlow / Keras CNN	No .pkl/.h5, no train script, no dataset in repo	absent	Not implemented	LOW
NLP	Keyword / substring psychological detector	Urgency, fear, OTP, countdown, greed, secrecy, Hinglish phrases	backend/app/services/psychology.py	Implemented	HIGH
NLP	Language heuristic	Devanagari count + Hinglish tokens → en / hi / en-hi	backend/app/services/content_normalizer.py	Implemented	MEDIUM
NLP	spaCy / transformers / BERT	Not in requirements or imports	absent	Not implemented	LOW
Vision	OpenCV QRCodeDetector + NumPy	Decode QR URLs from uploaded images; does not visit destination	backend/app/services/qr_analyzer.py	Implemented	HIGH
Vision	Pillow	Open screenshot bytes if OCR path runs	backend/app/routes/analyze.py	Implemented	MEDIUM
Vision	Tesseract / pytesseract	Optional OCR on screenshots; not in requirements.txt	analyze.py try/except import	Partial	MEDIUM
Browser	Chrome Extension Manifest V3	Popup, service worker, content script, settings, welcome	extension/manifest.json and subfolders	Implemented	HIGH
Browser	chrome.storage (local/session)	API URL, auto-scan, cache of scan results	extension/utils/api.js, background/	Implemented	HIGH
Cybersecurity	SSRF guard	Blocks fetch to localhost, private IPs, cloud metadata	backend/app/services/ssrf.py	Implemented	HIGH
Cybersecurity	Parameterized SQL (? placeholders)	All sqlite3 execute calls use bound parameters	connection.py, routes/, reputation.py	Implemented	HIGH
Cybersecurity	bcrypt via passlib	Password hashing at signup/login	backend/app/services/auth.py, routes/user.py	Implemented	HIGH
Cybersecurity	JWT HS256 (python-jose)	Bearer token; sub=username; login/signup pass 7-day expiry	auth.py, user.py, ProtectedRoute	Implemented	HIGH
Cybersecurity	CORS allow_origins= ['*']	Dev convenience; credentials=True with * is weak	backend/app/main.py	Partial	HIGH
Cybersecurity	Report flood caps	1 same URL / 24h and 20 reports / day per user	backend/app/services/reputation.py	Implemented	MEDIUM
Cybersecurity	Global API rate limiting	No SlowAPI/middleware in main.py	absent	Planned	MEDIUM
Cybersecurity	Upload size cap 5 MB	QR and screenshot endpoints	backend/app/routes/analyze.py	Implemented	MEDIUM

Domain	Technology	Actual use	Files / location	Status	Learning priority
APIs	REST JSON (FastAPI)	Primary integration for web + extension	backend/app/main.py routers	Implemented	HIGH
APIs	Google Safe Browsing v4	If GOOGLE_SAFE_BROWSING_API_KEY set in .env	backend/app/services/external_intel.py	Partial (key-dependent)	HIGH
APIs	OpenPhish feed	Fallback feed when GSB key not used / as coded fallback	external_intel.py OPENPHISH_FEED	Implemented	MEDIUM
APIs	RDAP / DNS	Domain existence and approximate age	external_intel.py	Implemented	MEDIUM
AuthZ	HTTPBearer optional vs required	Scans public; reports require login; profile requires JWT	analyze.py, reports.py, scans.py, user.py	Implemented	HIGH
Testing	pytest	API, psychology, URL, multi-channel tests	backend/tests/*.py, requirements.txt	Implemented	HIGH
Dev tools	Git	Version control of this repository	.git, .gitignore	Implemented	HIGH
Dev tools	npm / Node	Frontend install and Vite scripts	frontend/package.json	Implemented	MEDIUM
Deployment	Local uvicorn + Vite only	No Dockerfile, no compose, no cloud config in repo	README.md	Partial (local only)	MEDIUM
Mobile	Native Android SMS ingest	Stub / planned path	backend/app/services/mobile_source.py	Planned	LOW

3. Technology categories

A. Frontend

React 19, React Router 7, Axios, Lucide, Recharts, Vite 8, Tailwind 4, oxlint.

B. Backend

Python, FastAPI, Uvicorn, Pydantic, python-dotenv, requests, python-multipart, Pillow. SQLAlchemy is in requirements.txt and backend/app/models/ but **live routes use sqlite3**, not the ORM.

C. Database

SQLite file backend/threats.db via sqlite3. **PostgreSQL is planned, not live.** Settings.database_url in config.py defaults to sqlite:///./threats.db and is **not** what connection.py uses (that module builds a filesystem path to threats.db).

D. AI / machine learning

There is **no trained classifier**. Detection is **heuristics + optional threat intel + weighted fusion** in risk_engine.fuse. The fusion key "m1" is currently a **copy of the URL risk score** (content_analyzer.py).

E. NLP / text analysis

Substring / keyword matching in psychology.py (English + some Hinglish). Language tag in content_normalizer.detect_language (Devanagari count + a few tokens). **Not** transformer NLP.

F. Browser extension

Manifest V3 Chrome extension: service worker, content script, popup, settings. Talks to the same FastAPI process.

G. Cybersecurity

SSRF block on outbound fetches, parameterized SQL, bcrypt, JWT, trusted-domain allowlist, 5 MB upload cap, community report caps. CORS is open (*). No global API rate limit.

H. APIs

REST JSON. Optional Google Safe Browsing. OpenPhish feed fallback. DNS + RDAP. Frontend Axios; extension `fetch`.

I. Authentication / authorization

Signup/login JWT. Dashboard routes wrapped in `ProtectedRoute`. Scan APIs are **usable without login**. Community report requires login.

J. Development tools

Git, npm, Vite, oxlint, python-dotenv, uvicorn.

K. Testing

pytest modules: `test_api.py`, `test_threat_detection.py`, `test_psychology.py`, `test_multichannel.py`. **No frontend test suite** in `package.json`.

L. Deployment

Local only (Uvicorn :8000, Vite :5173). No Dockerfile in the repo.

M. Version control

Git. `.gitignore` excludes `.env`, `venv`, and typical junk. Treat `backend/.env` as secret (Safe Browsing key if present).

4. Explain every major technology

4.1 React

1. **What is it?** A JavaScript library for building the dashboard UI as components.
2. **Why?** Interactive scanner, history, charts, and auth screens without full page reloads.
3. **Where?** `frontend/src/` — `App.jsx`, `pages/Scanner.jsx`, `Dashboard.jsx`, `History.jsx`, `Reports.jsx`, `Statistics.jsx`, `Login.jsx`, `Signup.jsx`, `context/AuthContext.jsx`.
4. **Problem solved?** Users need a usable console, not only a raw JSON API.
5. **Concepts:** components, JSX, props, state, hooks (`useState`, `useEffect`), context, conditional rendering.
6. **Learn:** build one page that calls `analyzeContent` and shows `risk_score`.
7. **Jury:** Why React? What is a SPA? How does `ProtectedRoute` work?

4.2 JavaScript (ES modules)

The frontend and extension are JS modules ("type": "module" in `package.json` and extension service worker). Learn: `import / export`, `promises/ async`, `fetch` vs `AXIOS`, `localStorage`.

4.3 React Router DOM 7

Maps URLs to pages in `App.jsx`. Public: `/`, `/login`, `/signup`. Protected: `/dashboard`, `/scanner`, `/history`, `/reports`, `/statistics`, `/privacy`, `/settings`, `/advisor`. `/scan-result` uses `AdaptiveLayout` (works logged-in or out).

Jury: Difference between client routing and FastAPI routing.

4.4 Axios

`frontend/src/services/api.js` — `baseUrl` from `VITE_API_URL` or `http://localhost:8000`. Request interceptor adds `Authorization: Bearer` from `localStorage` key `cg_token`.

Discrepancy: Vite is bound to `127.0.0.1` (`vite.config.js`) while `.env.example` still says `localhost`. On this machine IPv6 `localhost` has bitten the team before — prefer `127.0.0.1`.

4.5 Tailwind CSS 4

Utility classes on JSX. Plugin `@tailwindcss/vite`. No separate `tailwind.config` required in this setup.

4.6 Vite 8

Dev server and bundler. Scripts: `dev`, `build`, `preview`, `lint`.

4.7 Recharts

Charts fed by `/api/v1/stats/*` JSON. If the API is down, the dashboard cannot invent numbers — it should fail visibly.

4.8 FastAPI

1. **What?** Python web framework for REST APIs with automatic OpenAPI at `/docs`.
2. **Why?** Typed request models, async endpoints, easy routers.
3. **Where?** `backend/app/main.py` mounts `threat_detection`, `user`, `stats`, `analyze`, `reports`, `scans`. Title: "CyberGuard API" version **2.0.0**.
4. **Solves:** one HTTP contract for React and the extension.
5. **Concepts:** router, path/query/body, `Depends`, `HTTPException`, CORS middleware, Pydantic.
6. **Learn:** add a GET that returns `{status: ok}` and call it from Axios.
7. **Jury:** Why not Flask? (Flask is **not** in this repo.) What is OpenAPI? What is CORS?

4.9 Uvicorn

ASGI server. Run: `python -m uvicorn app.main:app --host 0.0.0.0 --port 8000` from `backend/` with `venv` on.

4.10 Pydantic

Validates JSON bodies (`UserRegister`, `ContentRequest`, `URLCheckRequest`). Invalid bodies become 422, not silent bad data.

4.11 SQLite

1. **What?** File-based relational SQL database.
2. **Why (today)?** Zero-ops persistence for a local/demo app.
3. **Where?** `backend/app/database/connection.py` → `backend/threats.db`. `init_db()` on API startup.
4. **Solves:** remember users, URL checks, content scans, extracted links, community reports, domain reputation, scan feedback.
5. **Concepts:** tables, INTEGER PK AUTOINCREMENT, UNIQUE, FOREIGN KEY, indexes, `PRAGMA foreign_keys = ON`, parameterized queries.
6. **Learn:** draw the ER diagram in section 14 and write one `SELECT` with a JOIN.
7. **Jury:** Why not MongoDB? Why not PostgreSQL yet? What happens if `threats.db` is locked or missing? (`init_db` recreates schema; data loss if the file is deleted.)

4.12 PostgreSQL (PLANNED)

Not running. Do not say “we use PostgreSQL” unless you show a live connection. v2 argument: concurrent writers, managed hosting, backups, roles.

4.13 python-jose JWT + passlib bcrypt

- Passwords stored as `users.password_hash`.
- Token payload `{"sub": username, "exp": ...}`.
- Algorithm **HS256**, secret from `SECRET_KEY` env (defaults are **dev secrets** — say this in viva).
- Signup/login pass `expires_delta=timedelta(days=7)`.
- `ACCESS_TOKEN_EXPIRE_MINUTES = 30` in `auth.py` is **unused** when those routes pass an explicit delta.

4.14 Weighted fusion “AI”

See sections 9 and 12. This is the thing the team must explain better than “we used a neural network.”

4.15 OpenCV + NumPy

QR decode only. `cv2.QRCodeDetector`. Only payloads starting with `http` are kept. **Does not open the QR destination**. Then the URL goes through `analyze_content`.

4.16 Chrome Extension (Manifest V3)

See section 16.

4.17 pytest

Backend tests hit FastAPI TestClient and service functions. Frontend has no Jest/Vitest script.

4.18 Git

Team must know: what is committed, what is secret, how to explain a diff of `risk_engine.py`.

4.19 requests / external intel

DNS (`socket.getaddrinfo`), RDAP domain age, Google Safe Browsing if key present, otherwise OpenPhish feed (`OPENPHISH_FEED`). Timeouts and in-memory caches. **Fail soft:** if the internet is down, heuristics still return a score.

Docstring discrepancy: `external_intel.py` module comment mentions URLhaus; the constant in code is **OpenPhish**. In viva, describe **the code**.

4.20 SQLAlchemy models (unused on the live path)

`backend/app/models/user.py`, `url_check.py`, `threat.py` exist. Routers never import them for queries. Honest line: "ORM models were sketched; production path is `sqlite3`."

5. Team learning roadmap

LEVEL 1 — MUST KNOW (everyone)

The product in one sentence: a multi-channel scam intelligence API plus React dashboard plus Chrome extension that scores links and pasted messages using heuristics, optional threat intel, psychology keywords, community reports, and a weighted fusion engine.

Must know without code:

- Ports: API **8000**, UI **5173**
- Two scan pipelines: `POST /api/v1/threat/check-url` (URLChecker + `url_checks` table) and `POST /api/v1/analyze/content` (full fusion + `content_scans`)
- Risk 0–100; LOW < 30, MEDIUM < 50, HIGH < 70, else CRITICAL
- Psychology **cannot** alone produce HIGH/CRITICAL (cap 35)
- `model_confidence` is **signal agreement, not accuracy**
- Database is **SQLite**, not live Postgres
- No trained CNN/sklearn model file
- Extension does not read passwords, OTP, cookies, or phone SMS

React — levels

- MUST: components, props, state, hooks, Axios calls, conditional rendering, `localStorage` token
- SHOULD: `AuthContext`, `ProtectedRoute`, error messages in `getErrorMessage`
- JURY: SPA, why JWT in header, what happens on 401

FastAPI — levels

- MUST: route, JSON body, status codes, `/docs`
- SHOULD: routers, Pydantic, optional Bearer, CORS
- JURY: ASGI, why OpenAPI, difference between 400/401/422/500

SQLite — levels

- MUST: table vs row, PK, `users` / `content_scans` / `user_reports`
- SHOULD: FK `extracted_links.scan_id`, indexes, parameterized SQL
- JURY: SQLite vs Postgres vs Mongo; file locking; backups

Fusion engine — levels

- MUST: ten signal keys and `DEFAULT_WEIGHTS`
- SHOULD: psych-only cap, `_level()` thresholds, confidence formula `min(95, 35 + fired*10)`
- JURY: "m1": `url_risk` means no separate ML model; weights are **starting defaults, not proven-optimal** (comment in `risk_engine.py`)

Psychology — levels

- MUST: keyword categories and “not a verdict by itself”
- SHOULD: `analyze_message` vs `analyze_text` (URL path scales psych by 0.4, cap 40)
- JURY: substrings false positives (“urgent” in news)

Extension — levels

- MUST: MV3 pieces, default API `http://127.0.0.1:8000`
- SHOULD: message passing, overlay on HIGH/CRITICAL, visible-links only
- JURY: `host_permissions`, why SMS impossible in Chrome

Security — levels

- MUST: `bcrypt`, JWT, `?` SQL, SSRF helper
- SHOULD: CORS `*`, no global rate limit, report caps
- JURY: SSRF to 169.254.169.254; XSS (React escapes text; do not `dangerouslySetInnerHTML` — not used in core pages inspected)

6. Role-based learning plan

Team member names are not in the repository. Do not invent names. Use roles.

Role	Primary domain	Secondary	Must learn	Viva priority
Backend Developer	FastAPI, services, sqlite3	Auth, intel APIs	Python, FastAPI, Pydantic, parameterized SQL, JWT, <code>analyze_content</code> flow	HIGH
Frontend Developer	React dashboard	UX copy for Simple View	React, Router, Axios, Tailwind, AuthContext, Scanner tabs	HIGH
ML / Detection Developer	Fusion + heuristics + psychology	QR/OpenCV	<code>url_checker.py</code> , <code>risk_engine.py</code> , <code>psychology.py</code> , honesty about no trained model	HIGH
Database Developer	SQLite schema	Future Postgres	<code>connection.py</code> tables, indexes, what is NOT migrated	MEDIUM
Extension Developer	Manifest V3	Privacy	service worker, content script, <code>utils/api.js</code> , permissions	HIGH
Security / Research Lead	SSRF, auth, threat intel, limitations	Roadmap	<code>ssrf.py</code> , CORS gap, GSB vs OpenPhish, false positive policy	HIGH

Shared for all roles: one full request trace (section 8), risk-score story (section 12), cheat sheet (section 21).

7. Project architecture (as implemented)

This is **not** “URL in → CNN out.” Two client types share one API process.

```

USER
| types URL / pastes message / uploads QR | visits a webpage
v
REACT DASHBOARD (Vite :5173)          CHROME EXTENSION (MV3)
| JSON + optional Bearer JWT          | fetch JSON (same endpoints)
v
FASTAPI (Uvicorn :8000) --- CORS * ---
| routers: /api/v1/analyze/* /threat/* /user/* /stats/* /scans /report
v
OPTIONAL AUTH (verify JWT → user_id or None)
v
CONTENT NORMALIZER (source_type, extract URLs, language heuristic)
+-----+-----+
v           v           v
URLChecker      psychology      brand + sender
(heuristics,    (keywords)      + community reputation
redirects,
ads/SSRF-gated
fetch, DNS,
RDAP, GSB/OpenPhish)
v
RISK ENGINE fuse(signals, DEFAULT_WEIGHTS)
| psych-only cap 35; levels LOW/MEDIUM/HIGH/CRITICAL
v
explanation.py + Simple View + Technical View
v
SQLITE  content_scans + extracted_links (analyze path)
        url_checks + threats           (legacy check-url path)
v
JSON RESPONSE
v
UI: score, level, reasons, Simple View, Technical View
EXT: badge, toast, overlay if HIGH/CRITICAL

```

What moves on each arrow

From → To	Data	Format	Why
Scanner → FastAPI	source_type, text, urls[], optional sender	JSON POST	User content must be normalized server-side
Extension popup → FastAPI	page URL or {source_type:"webpage", urls:[...]}	JSON POST	Same engine as dashboard
FastAPI → URLChecker	one URL string	Python str	Link reputation / heuristics
FastAPI → psychology	raw text	str	Social-engineering language
URLChecker → external_intel	host/URL	HTTPS/DNS	Optional live intel
URLChecker → ad_signals / redirects	URL	HTTP GET only after assert_public_http_url	Expand short links / look at page signals without hitting private IPs
All signals → fuse	dict of 0–100 floats	Python dict	Single score
fuse → sqlite	score, level, source_type, urls	SQL INSERT	History / dashboard
FastAPI → clients	risk_score, risk_level, signals, weights, explanation, simple_view, technical_view	JSON	Human + technical UI

8. Trace one complete request

Example A — user pastes a URL in the dashboard Scanner (URL tab).

Today the Scanner uses `analyzeContent` (`POST /api/v1/analyze/content`), not only the legacy checker. Extension page scan still uses `checkUrl` → `POST /api/v1/threat/check-url` . **Know both.**

Path A — unified analysis (dashboard message/URL tabs)

Step	File	Function	Input	Output
1. User types URL	<code>frontend/src/pages/Scanner.jsx</code>	form submit	string	state
2. HTTP	<code>frontend/src/services/api.js</code>	<code>analyzeContent</code>	<code>{source_type, text, urls}</code> + Bearer if logged in	Axios POST
3. Route	<code>backend/app/routes/analyze.py</code>	<code>analyze_any</code>	<code>ContentRequest</code>	400 if empty; else service
4. Optional user	<code>analyze.py</code>	<code>_user_id</code>	JWT	<code>users.id</code> or None
5. Pipeline	<code>content_analyzer.py</code>	<code>analyze_content</code>	source, text, urls, sender, language, user_id	payload dict
6. Normalize	<code>content_normalizer.py</code>	<code>normalize_content</code>	raw text/urls	<code>raw_text</code> , <code>urls[]</code> , <code>domains</code> , <code>language</code> , <code>source_type</code>
7. Extract links	<code>url_extractor.py</code>	<code>extract_urls</code>	text	URL list
8. Each link	<code>url_checker.py</code>	<code>URLChecker.analyze</code>	URL	<code>risk_score</code> , tags, details, simple/technical views
9. Psych	<code>psychology.py</code>	<code>analyze_message</code>	raw_text	score 0–100, tags, findings
10. Brand	<code>brand_detector.py</code>	<code>detect_impersonation</code>	text + urls + extra flags	score, impersonated
11. Sender	<code>sender_analyzer.py</code>	<code>analyze_sender</code>	sender dict + text	score, mismatch
12. Community	<code>reputation.py</code>	<code>get_reputation</code>	domain	<code>reputation_score</code> from <code>domain_reputation</code>
13. Fuse	<code>risk_engine.py</code>	<code>fuse</code>	ten signals	<code>risk_score</code> , level, confidence, weights
14. Explain	<code>explanation.py</code> , <code>warnings.py</code>	<code>build_explanation</code> , <code>build_simple_view</code> , <code>build_technical_view</code>	payload	user-facing text
15. Persist	<code>content_analyzer.py</code> (DB inserts)	sqlite	scan row + extracted_links	<code>content_scans.id</code>
16. Response	FastAPI	JSON	—	frontend state
17. UI	<code>ScanResult.jsx</code> / Scanner result panel	render	JSON	score, Simple View, Technical View

Fusion inputs actually passed (`content_analyzer.py`):

```

url      = max URLChecker scores (url_risk)
domain   = _domain_risk(top link)
ml       = url_risk          # COPY – not a separate model
nlp      = psych["score"]    # SAME number as psychological
psychological = psych["score"]
threat_intel = intel from URL details / GSB
brand     = brand detector score
redirect  = redirect_risk
sender    = sender_info["score"]
community = reputation_score

```

Path B — extension / legacy URL check

Step	File	Function
Popup or auto-scan	extension/popup/popup.js, background/service-worker.js	checkUrl
HTTP	extension/utils/api.js	POST /api/v1/threat/check-url {url, view:"both"}
Route	threat_detection.py	check_url — prepends https:// if missing scheme
Engine	URLChecker.analyze only (not full ten-signal fuse)	heuristics + psych via analyze_text (scaled)
Persist	INSERT url_checks ; if HIGH/CRITICAL also threats	sqlite
UI	badge / toast / overlay	content script

Jury trap: “Is every scan using the fusion engine?” **No.** check-url is the older URLChecker path. Dashboard multi-channel uses analyze/content.

9. AI / ML explanation (honest)

Question	Answer from code
What model is used?	No trained model. Weighted sum in fuse().
Why selected?	Explainable, works without a labelled dataset in-repo, suitable for a prototype.
Input features?	Heuristic URL features (TLD, brand typo, keywords, IP URL, shortener, domain age, DNS, intel, ads) + psych tags + sender + community counts. Not a numeric sklearn feature vector.
How is data processed?	Normalize → per-link URLChecker.analyze → keyword psych → fuse.
How is the model trained?	It is not. No train script, no dataset file, no train/test split.
How is it loaded?	N/A. Python imports risk_engine.py.
Prediction?	int(round(weighted sum)) then optional cap 35.
Output meaning?	risk_score 0–100 evidence score; risk_level band; model_probability equals the score (not a calibrated probability).
Confidence?	min(95, 35 + 10 * count(signals >= 20)) — agreement, not accuracy .
False positives?	Trusted domain list; psych-only cap; Simple View explains “marketing language”. Still possible on new legitimate domains / keyword collisions.
False negatives?	New brands, obfuscated text, image-only lures without QR, intel timeout.
Evaluation metrics?	None in repo (no F1/accuracy/ROC). pytest checks behaviour, not ML quality.

Question	Answer from code
Production-ready?	Prototype / demo. Weights are documented as starting defaults.

CNN / deep learning / sklearn: not implemented. NumPy is used to reshape image bytes for OpenCV, not for neural nets.

10. Psychological threat detection

Implemented in `backend/app/services/psychology.py` as **case-insensitive substring** checks.

Categories (names in `CATEGORIES`): urgency, fear/coercion, authority, financial pressure, credential pressure (OTP/KYC), countdown, greed/reward bait, secrecy, emotional manipulation. Patterns include English and some Hinglish (`mandatory kyc`, `please beta`, `inaam`, etc.).

`analyze_message`: add `points` per matched category, cap 100.

`analyze_text` (used by URLChecker): `min(int(score * 0.4), 40)` so page-text psych cannot dominate a URL-only check.

Fusion guard: if psychological ≥ 20 and no strong url/domain/intel/brand/community signal (≥ 40), **score is capped at 35** → at most MEDIUM.

Difference vs URL-only phishing: URL checkers look at the destination. This looks at **how the message tries to rush or scare the human**. A legitimate bank domain can still show psych tags; a message with no link can still score MEDIUM from language alone.

Not implemented: trained emotion classifier, transformer sentiment, full Hindi morphology, conversation context across messages.

PLANNED / not this file: treating psych as a legal verdict; auto-reading WhatsApp.

11. Multi-channel analysis

`VALID_SOURCES` in `content_normalizer.py`: `url`, `text`, `email`, `whatsapp`, `sms`, `webpage`, `screenshot`, `qr`.

Channel	Current support	How it works	Technology	Limitation
URL	Implemented	Dashboard + <code>analyze/url</code> + <code>check-url</code> + extension	URLChecker + optional intel	Two pipelines (fusion vs URLChecker)
Pasted text	Implemented	Scanner paste tab → <code>source_type: text</code>	extractor + psych + fusion	User must paste; no inbox access
Email text	Implemented	User pastes headers/body + optional sender object	<code>analyze/email</code> , <code>sender_analyzer</code>	No Gmail OAuth
WhatsApp	Partial	Paste conversation text, <code>source_type: whatsapp</code>	same as text	Does not read WhatsApp
SMS	Partial	Paste SMS text	same as text	No native SMS; <code>mobile_source.py</code> planned
Webpage	Implemented (extension)	Visible <code>http(s)</code> links + current URL	content script + API	Does not rewrite hrefs; no password fields
QR	Implemented	Upload image → OpenCV decode → analyze URLs	opencv-python-headless	Only <code>http(s)</code> payloads; needs a readable QR

Channel	Current support	How it works	Technology	Limitation
Screenshot	Partial	QR always; OCR if pytesseract + Tesseract installed	Pillow + optional Tesseract	Tesseract not in requirements.txt ; error tells user to paste text
Native Android SMS	Planned	mobile_source.py	—	Do not demo as working

12. Risk score

Range: 0–100 integer after rounding.

Formula:

```
raw =  $\sum$  signal[k] * DEFAULT_WEIGHTS[k]
```

```
then score = min(round(raw), 100)
```

```
then if psych-only: score = min(score, 35).
```

Default weights (sum 1.0): url 0.22, domain 0.16, ml 0.10, nlp 0.10, psychological 0.10, threat_intel 0.10, brand 0.08, redirect 0.04, sender 0.05, community 0.05.

Is it ML probability? No. model_probability is set to the same fused score.

What does 35/100 mean? Borderline MEDIUM (MEDIUM starts at 30). If it was psych-only, 35 is the **ceiling**, not "35% chance of fraud."

Levels: LOW 0–29, MEDIUM 30–49, HIGH 50–69, CRITICAL 70–100 (_level in risk_engine.py). URLChecker has its own score/level for path B.

Confidence ≠ accuracy. Many signals $\geq 20 \rightarrow$ higher confidence even if all are wrong together.

Thresholds: chosen as documented starting defaults in source comments — **not** from a ROC curve in this repo.

Scam risk: same integer as risk_score (scam_risk: score).

13. Security

Control	Status	Where
Input validation	Implemented	Pydantic models; empty URL/message 400; username ≥ 3 , password ≥ 6
URL scheme	Implemented	check_url prefixes https:// if missing
SSRF prevention	Implemented on outbound fetch	ssrf.assert_public_http_url — localhost, private, link-local, multicast, reserved, metadata IPs
SQL injection	Implemented	sqlite3 ? placeholders
XSS	Partial / framework default	React text interpolation; no sanitizer library; do not claim a dedicated XSS gateway
CSRF	Not implemented as tokens	JWT in Authorization header (not cookie-auth). CORS * is the real issue
Authentication	Implemented	bcrypt + JWT
Authorization	Partial	Reports/feedback/delete-all-scans need login; GET /scans is public ; analyze is optional auth
Password security	Implemented	bcrypt via passlib; plaintext never stored
Rate limiting	Partial	Report 24h/URL and 20/day; no global API limiter
API security	Partial	No API keys for scan; open CORS

Control	Status	Where
Secrets	Implemented pattern	<code>.env</code> + <code>python-dotenv</code> ; default SECRET_KEY is unsafe
CORS	Partial (weak)	<code>allow_origins=["*"]</code> , <code>allow_credentials=True</code>
HTTPS	Not enforced in app	Local HTTP demo
Upload cap	Implemented	5 MB
Privacy / minimization	Documented + extension copy	Extension description: no passwords/OTP/cookies/SMS; scans may store URL strings in SQLite
Trusted allowlist	Implemented	<code>TRUSTED_DOMAINS</code> in <code>url_checker.py</code>

14. Database

Engine: SQLite. **File:** `backend/threats.db` . **Created by:** `init_db()` .

Table	Purpose	Important columns	PK	FKs / relationships
users	Accounts	username UNIQUE, email UNIQUE, password_hash, created_at	id	url_checks.user_id references users (nullable)
url_checks	Legacy URL scan log	url, risk_level, risk_score, ml_confidence, timestamp, user_id	id	user_id → users.id
threats	High/critical URL events from check-url	threat_type, threat_data, severity, last_updated	id	none
content_scans	Multi-channel scan logs	user_id, source_type, url, risk_level, risk_score, scam_risk, confidence, created_at	id	user_id not declared FK in SQL
extracted_links	Links found in a content scan	scan_id, url, domain, risk_score, classification	id	scan_id → content_scans.id
user_reports	Community labels	user_id, url, domain, user_label, reason, created_at	id	user_id not declared FK
domain_reputation	Aggregated votes	domain, scam_reports, risky_reports, safe_reports, last_reported	domain (text PK)	none
scan_feedback	Helpful flag	scan_id, helpful, created_at	id	scan_id not declared FK

Indexes: `url_checks(url, user_id, timestamp)`; `reports(url, user_id, domain, created_at)`; `scans(created_at, source_type, risk_score)`; `extracted_links(domain, url, classification)`.

ER (actual)

```
users 1---0..* url_checks
users      (logical) 0..* content_scans, user_reports  [no FK in CREATE TABLE]
content_scans 1---0..* extracted_links
domain_reputation 1---0..* user_reports.domain  [logical only]
```

SQLAlchemy models (`User` , `UrlCheck` , `Threat`) do not define the 2.0 tables.

15. API catalogue

Auth column: Public = no token required. Optional = token used only to attach user_id. Required = 401 without valid JWT.

Method	Endpoint	Purpose	Request	Auth	Used by
GET	/	Health + endpoint list	—	Public	browsers, debugging
GET	/health	{status: ok}	—	Public	frontend healthCheck , extension
POST	/api/v1/user/signup	Register	{username,email,password}	Public	Signup.jsx
POST	/api/v1/user/login	JWT	{username,password}	Public	Login.jsx, extension login
GET	/api/v1/user/profile	Current user	Bearer	Required	AuthContext
POST	/api/v1/threat/check-url	URLChecker pipeline	{url, view}	Public	UrlScanner, extension checkUrl
POST	/api/v1/threat/check-message	Wrapper over analyze_content text	{message, url?}	Public	api.js checkMessage
GET	/api/v1/threat/stats	Counts from url_checks/threats	—	Public	Dashboard
GET	/api/v1/threat/recent-urls	Recent url_checks	limit	Public	History-related
POST	/api/v1/analyze/content	Full fusion pipeline	{source_type,text,urls,sender,language}	Optional	Scanner, extension analyzeContent
POST	/api/v1/analyze/url	Fusion with source url	same	Optional	Scanner URL helper
POST	/api/v1/analyze/email	Fusion source email	same	Optional	Scanner email tab
POST	/api/v1/analyze/qr	multipart image	file ≤ 5 MB	Optional	Scanner QR
POST	/api/v1/analyze/screenshot	QR + optional OCR	file ≤ 5 MB	Optional	Scanner screenshot
POST	/api/v1/report	Community label scam/risky/safe	JSON url/domain/label/reason	Required	Reports.jsx
GET	/api/v1/reputation/domain/{domain}	Vote tallies	—	Public	Reports / result UI
POST	/api/v1/feedback	helpful flag	scan_id, helpful	Public (check route)	Scan result
GET	/api/v1/reports/mine	Current user reports	Bearer	Required	Reports.jsx
GET	/api/v1/scans	Recent content_scans	limit	Public	History.jsx
GET	/api/v1/scans/{id}	One scan	—	Public	History detail
DELETE	/api/v1/scans/{id}	Delete one	Bearer (see scans.py)	Required for destructive	History
DELETE	/api/v1/scans	Clear history	Bearer	Required	History
GET	/api/v1/stats/overview	Aggregates	—	Public	Dashboard
GET	/api/v1/stats/threat-types	from threats table	—	Public	Statistics
GET	/api/v1/stats/risk-distribution	buckets	—	Public	Statistics
GET	/api/v1/stats/daily-summary	per day	—	Public	Statistics
GET	/api/v1/stats/sources	source_type counts	—	Public	Dashboard

Method	Endpoint	Purpose	Request	Auth	Used by
GET	/api/v1/stats/timeline	time series	—	Public	Statistics
GET	/api/v1/stats/reports-summary	community totals	—	Public	Reports

Interactive docs: <http://127.0.0.1:8000/docs> (Swagger) and </redoc>.

16. Browser extension

Status: Implemented (Manifest V3), folder `extension/`.

Piece	File	Role
Manifest	<code>extension/manifest.json</code>	MV3, version 1.2.0, name CyberGuard
Service worker	<code>background/service-worker.js</code>	auto-scan, badge, cache
Content script	<code>content/content.js</code>	toast, HIGH/CRITICAL overlay, collect visible links
Popup	<code>popup/popup.html</code> + <code>popup.js</code>	scan page / visible links
Settings	<code>settings/</code>	API base URL, auto-scan
API helper	<code>utils/api.js</code>	<code>checkUrl</code> , <code>analyzeContent</code> , login, cache
Config	<code>utils/config.js</code>	defaults
Welcome	<code>welcome/</code>	first-run

Permissions: `tabs`, `storage`. **Host permissions:** `127.0.0.1`, `localhost`, `http://`, `https://` (needed to call local API and to run on pages).

Communication: `fetch` to FastAPI (same JSON as React). Results cached in `chrome.storage`. Content script ↔ worker via `chrome.runtime.sendMessage` (inspect those files for exact message types before viva).

Does not: read passwords, OTP, cookies, or phone SMS (stated in manifest description).

Requires: backend running; user may set API URL in settings if not `127.0.0.1:8000`.

17. Jury questions you must be ready for

Format: **Q / SHORT / DEEPER / WHERE.**

17.1 BASIC (20)

Q1. What is PhishShield AI / CyberGuard?

SHORT: A prototype that scores scam risk in links and pasted messages.

DEEPER: FastAPI engine + React console + MV3 extension; fusion of heuristics, intel, psychology, community.

WHERE: `README.md`, `main.py`.

Q2. What problem does it solve?

SHORT: People click scam links in SMS/WhatsApp/email, not only in the browser address bar.

DEEPER: Multi-channel paste + QR + visible page links.

WHERE: `Scanner.jsx`, `analyze.py`.

Q3. What is phishing?

SHORT: A social-engineering attack that tricks a human into acting (login, OTP, payment).

DEEPER: May use fake domains, stolen brands, or real-looking language.

WHERE: url_checker.py, psychology.py.

Q4. Frontend vs backend?

SHORT: React UI vs FastAPI JSON API.

DEEPER: UI never computes the official score; the server does.

WHERE: frontend/src, backend/app.

Q5. Why Python?

SHORT: Fast services, string/URL parsing, OpenCV, tests.

DEEPER: Team speed; not because Python is "more secure."

WHERE: backend/.

Q6. Why React?

SHORT: Component UI for scanner, charts, auth.

DEEPER: SPA with Router and Axios.

WHERE: frontend/package.json.

Q7. What is an API?

SHORT: HTTP contract: method + path + JSON.

DEEPER: Clients are React and the extension.

WHERE: main.py routers.

Q8. What is REST?

SHORT: Resource-style HTTP (GET scans, POST analyze).

DEEPER: We use JSON REST, not GraphQL.

WHERE: routes/.

Q9. What is JSON?

SHORT: The request/response body format.

DEEPER: Pydantic parses it.

WHERE: all POST routes.

Q10. What is SQLite?

SHORT: A SQL database in one file.

DEEPER: Good for demo; weaker for many concurrent writers.

WHERE: connection.py.

Q11. What is a primary key?

SHORT: Unique row id.

DEEPER: AUTOINCREMENT integers; domain_reputation uses domain text PK.

WHERE: connection.py.

Q12. What is a foreign key?

SHORT: A column pointing at another table's PK.

DEEPER: `extracted_links.scan_id` → `content_scans.id`; PRAGMA foreign_keys ON.

WHERE: `connection.py`.

Q13. HTTP vs HTTPS?

SHORT: HTTPS encrypts the pipe.

DEEPER: Local demo is HTTP; production must use HTTPS.

WHERE: local unicorn.

Q14. What is a JWT?

SHORT: Signed token proving login.

DEEPER: HS256, `sub` =username, sent as Bearer.

WHERE: `auth.py`.

Q15. GET vs POST?

SHORT: GET reads; POST sends a body (scan, login).

DEEPER: Scans are POST so URLs are not stuck in server logs as query strings as often (still stored in DB).

WHERE: `analyze.py`.

Q16. What is a risk score?

SHORT: 0–100 fused evidence, not a probability of crime.

DEEPER: See section 12.

WHERE: `risk_engine.py`.

Q17. Chrome extension vs website?

SHORT: Extension runs in the browser on other sites; website is our React app.

DEEPER: MV3 service worker + content script.

WHERE: `extension/`.

Q18. What is CORS?

SHORT: Browser rule for cross-origin API calls.

DEEPER: We currently allow `*`.

WHERE: `main.py`.

Q19. What is bcrypt?

SHORT: Password hashing.

DEEPER: One-way; salt included; not reversible encryption.

WHERE: `auth.py`.

Q20. How do you start the project?

SHORT: `unicorn :8000` and `npm run dev :5173`.

DEEPER: `venv`, `.env`, SQLite init on startup.

WHERE: `README.md`.

17.2 INTERMEDIATE (20)

Q21. Why FastAPI instead of Flask?

SHORT: FastAPI is what we implemented; Flask is not in the repo.

DEEPER: Type hints, OpenAPI, async-ready, Pydantic validation.

WHERE: requirements.txt.

Q22. Why SQLite instead of PostgreSQL?

SHORT: Local prototype speed.

DEEPER: Postgres is the likely v2 store for concurrency and hosting.

WHERE: connection.py vs config.py.

Q23. How is SQL injection prevented?

SHORT: Bound parameters, never f-string SQL with user text.

DEEPER: `cursor.execute("... ?", (value,))`.

WHERE: user.py, reputation.py.

Q24. How does login work end-to-end?

SHORT: POST login → bcrypt verify → JWT → localStorage → Axios header.

DEEPER: profile uses HTTPBearer.

WHERE: user.py, api.js, AuthContext.jsx.

Q25. Why can scans be anonymous?

SHORT: Lower friction for checking a link.

DEEPER: Reports are authenticated to reduce vandalism.

WHERE: analyze.py optional_bearer; reports.py.

Q26. What is SSRF and how do you block it?

SHORT: Tricking the server to fetch internal URLs.

DEEPER: Block private IPs and metadata hosts before GET.

WHERE: ssrf.py.

Q27. How are shortened URLs handled?

SHORT: Redirect expansion with public-URL checks.

DEEPER: `redirects.py` + shortener list in `url_checker`.

WHERE: redirects.py.

Q28. What does Simple View vs Technical View mean?

SHORT: Plain language vs metrics.

DEEPER: `warnings.build_simple_view` / `build_technical_view`.

WHERE: warnings.py, ScanResult.jsx.

Q29. How does community reputation work?

SHORT: Logged-in users mark scam/risky/safe; domain tallies feed a fusion signal.

DEEPER: 1 URL/24h and 20/day; not proof.

WHERE: reputation.py.

Q30. Why OpenCV?

SHORT: Decode QR images.

DEEPER: Not a phishing CNN.

WHERE: qr_analyzer.py.

Q31. What if Google Safe Browsing is missing?

SHORT: Key empty → skip GSB; use other signals + OpenPhish path.

DEEPER: Fail soft.

WHERE: external_intel.py.

Q32. What is Pydantic validation?

SHORT: Reject malformed JSON before business logic.

DEEPER: 422 vs 400 we raise ourselves.

WHERE: ContentRequest.

Q33. How does the dashboard get chart data?

SHORT: GET `/api/v1/stats/*` then Recharts.

DEEPER: Aggregates from SQLite, not from a warehouse.

WHERE: stats.py, Statistics.jsx.

Q34. Manifest V3 vs V2?

SHORT: V3 uses a service worker, not a persistent background page.

DEEPER: Our `background.service_worker` field.

WHERE: manifest.json.

Q35. How do you stop report flooding?

SHORT: Per-user caps.

DEEPER: Not a substitute for global rate limits.

WHERE: reputation.py.

Q36. What is stored from a scan?

SHORT: source_type, url, scores, extracted links — not full message body in content_scans columns.

DEEPER: Read INSERT in content_analyzer; raw text may still appear in other fields depending on path — verify before claiming minimization.

WHERE: connection.py, content_analyzer.py.

Q37. Trusted domains?

SHORT: Allowlist reduces false positives on google.com etc.

DEEPER: Can hide attacks on compromised popular sites — limitation.

WHERE: TRUSTED_DOMAINS.

Q38. Two scan APIs?

SHORT: check-url vs analyze/content.

DEEPER: Extension page scan vs dashboard fusion.

WHERE: threat_detection.py vs analyze.py.

Q39. How are tests run?

SHORT: `pytest` in backend.

DEEPER: They assert API/service behaviour, not ML accuracy.

WHERE: backend/tests/.

Q40. What is an index?

SHORT: Speeds lookups on url, domain, timestamp.

DEEPER: `CREATE INDEX IF NOT EXISTS` in `init_db`.

WHERE: `connection.py`.

17.3 ADVANCED (20)

Q41. Is your system machine learning?

SHORT: Marketing "AI"; technically **rule fusion**, not a fitted model.

DEEPER: `"m1"` copies URL risk.

WHERE: `content_analyzer.py` line assigning `"m1": url_risk`.

Q42. Why not a CNN on screenshots?

SHORT: We did not train one; OCR is optional Tesseract.

DEEPER: Need labelled data, compute, and still fail on new kits.

WHERE: `analyze.py` screenshot handler.

Q43. How would you add a real sklearn model?

SHORT: Extract features, train, dump joblib, load in fuse as the ml signal.

DEEPER: Version the model; keep heuristics as fallback.

WHERE: `risk_engine.py` weights already have an `m1` slot.

Q44. Are weights scientifically optimal?

SHORT: No — comments say starting defaults.

DEEPER: Would need labelled outcomes and calibration.

WHERE: `risk_engine.py` docstring.

Q45. Confidence vs accuracy vs precision?

SHORT: Confidence here is agreement count.

DEEPER: Accuracy needs ground truth we do not store as a metric table.

WHERE: `fuse()`.

Q46. How do you calibrate probability?

SHORT: We do not. `model_probability` is the score.

DEEPER: Do not tell the jury "87% chance this is phishing."

WHERE: `fuse` return dict.

Q47. SQLite write concurrency?

SHORT: File DB can lock under parallel extension + dashboard.

DEEPER: Reason to move to Postgres.

WHERE: threats.db.

Q48. JWT HS256 vs RS256?

SHORT: We use a shared secret HS256.

DEEPER: RS256 would allow the API to verify with a public key; overkill for one process.

WHERE: auth.py ALGORITHM.

****Q49. CORS * with credentials True?****

SHORT: Browser-inconsistent / unsafe pattern.

DEEPER: Production: explicit origin list.

WHERE: main.py.

Q50. Authorization gap on GET /scans?

SHORT: History listing is public at the API.

DEEPER: UI may hide it behind login but the endpoint is still callable.

WHERE: scans.py `list_scans`.

Q51. How to prevent XSS in Simple View strings?

SHORT: React escapes; never inject HTML from reasons.

DEEPER: Reasons come from our templates + snippets of user text.

WHERE: warnings.py, ScanResult.jsx.

Q52. DNS rebinding vs your SSRF check?

SHORT: We resolve and block private IPs at check time.

DEEPER: Classic bypass is TOCTOU / rebinding — not fully closed. Honest limitation.

WHERE: ssrf.py.

Q53. Unicode / homograph domains?

SHORT: Partial via brand detector / parsing; not a full IDN homograph engine.

DEEPER: Do not claim Punycode mastery unless you show code.

WHERE: url_checker.py, brand_detector.py.

Q54. Hinglish coverage?

SHORT: A few substrings, not a language model.

DEEPER: Easy to evade with spelling changes.

WHERE: psychology.py.

Q55. Adversarial ML?

SHORT: Not applicable until we have a fitted model; attackers already evade keywords.

DEEPER: Obfuscation, images, new TLDs.

WHERE: psychology + url_checker.

Q56. How would you evaluate detection quality?

SHORT: Labelled dataset, precision/recall by channel, confusion matrix, live false-positive review.

DEEPER: Not in repo today.

WHERE: tests only.

Q57. Secrets in git?

SHORT: `.env` gitignored; defaults in code are weak.

DEEPER: Rotate any key that was ever pasted in chat.

WHERE: `.gitignore`, `auth.py` defaults.

Q58. Why optional `numpy`/`opencv` import errors?

SHORT: QR returns a clear error if OpenCV missing.

DEEPER: requirements include `opencv-python-headless` so install should work.

WHERE: `qr_analyzer.py`.

Q59. Stateless API + SQLite?

SHORT: API processes are stateless; state is the file DB.

DEEPER: Two unicorn workers would share the file with locking.

WHERE: `connection.py`.

Q60. What is explainability?

SHORT: We return weights, per-signal scores, tags, Simple View.

DEEPER: That is why fusion beats a black-box CNN for a student viva.

WHERE: `explanation.py`, fuse methodology string.

17.4 PROJECT-SPECIFIC (20)

Q61. Why is the API named CyberGuard?

SHORT: Historical code name; SIH title is PhishShield AI.

DEEPER: `FastAPI(title="CyberGuard API")`.

WHERE: `main.py`.

Q62. What does `ml_confidence` on `url_checks` mean?

SHORT: Column default 0.0; `check-url` insert may not set it.

DEEPER: Do not confuse with `model_confidence` from fusion.

WHERE: `connection.py`, `threat_detection` insert.

Q63. Does `check-message` use fusion?

SHORT: Yes — it calls `analyze_content`.

DEEPER: Unlike `check-url`.

WHERE: `threat_detection.py` `check_message`.

Q64. How is language detected?

SHORT: Devanagari count and a few Hinglish tokens.

DEEPER: Returns `hi`, `en-hi`, or `en`.

WHERE: `content_normalizer.py`.

Q65. Brand impersonation vs trusted domain?

SHORT: Brand detector looks at text/links; trusted list skips many URL heuristics.

DEEPER: A trusted domain can still carry a scam **message**.

WHERE: brand_detector.py, TRUSTED_DOMAINS.

Q66. What is `scam_risk`?

SHORT: Alias of fused score.

DEEPER: Not a second model.

WHERE: fuse().

Q67. Screenshot without QR?

SHORT: Tries Tesseract; if no text and no QR → 400 with paste instruction.

DEEPER: pytesseract not in requirements.

WHERE: analyze.py `analyze_screenshot`.

Q68. Does the extension send every URL on a page?

SHORT: Visible http(s) links / current page — not hidden password fields.

DEEPER: Popup copy says hrefs are not rewritten.

WHERE: popup.html, content.js.

Q69. Who can delete scan history?

SHORT: DELETE `/api/v1/scans` requires auth.

DEEPER: GET list does not.

WHERE: scans.py.

Q70. What is Advisor page?

SHORT: Frontend route `/advisor` — explain from the page, do not overclaim backend intelligence.

DEEPER: Open `Advisor.jsx` before viva and describe only what it renders.

WHERE: frontend/src/pages/Advisor.jsx.

Q71. config.py DATABASE_URL vs connection.py?

SHORT: Settings string is unused by get_db_connection.

DEEPER: Changing DATABASE_URL in env will **not** move SQLite unless you change connection.py.

WHERE: config.py, connection.py.

Q72. ACCESS_TOKEN_EXPIRE_MINUTES?

SHORT: 30 in auth.py but routes pass 7 days.

DEEPER: Dead constant.

WHERE: auth.py vs user.py.

Q73. OpenPhish vs URLhaus?

SHORT: Code fetches OpenPhish feed.

DEEPER: File header comment is stale if it says URLhaus.

WHERE: external_intel.py.

Q74. What tests cover psychology?

SHORT: `backend/tests/test_psychology.py`.

DEEPER: Keyword presence, not linguistic theory.

WHERE: tests/.

Q75. Can psychology alone be CRITICAL?

SHORT: No — cap 35 in fuse.

DEEPER: By design to cut marketing false positives.

WHERE: risk_engine.py.

Q76. What is `safe` in the JSON?

SHORT: `risk_level == "LOW"` on analyze payload.

DEEPER: LOW is not "proven safe."

WHERE: content_analyzer.py payload.

Q77. How are piracy sites treated?

SHORT: PIRACY_DOMAINS list in URLChecker.

DEEPER: Heuristic, not a court judgement.

WHERE: url_checker.py.

Q78. Feedback table?

SHORT: `scan_feedback` + POST `/api/v1/feedback`.

DEEPER: Does not retrain any model.

WHERE: reports.py, connection.py.

Q79. Biggest limitation?

SHORT: No trained detector, SQLite, open CORS, paste-only messaging, keyword evasion.

DEEPER: See section 18.

WHERE: whole repo.

Q80. What is version 2?

SHORT: Postgres, real ML slot, OCR packaged, origin-restricted CORS, global rate limit, Android SMS companion, calibrate scores.

DEEPER: Do not present v2 as done.

WHERE: mobile_source.py, FEATURE_ROADMAP.md if present.

17.5 TRICK QUESTIONS

T1. "Show me your CNN accuracy."

There is no CNN and no accuracy metric in the repo.

T2. "You use PostgreSQL, right?"

No. SQLite file `threats.db`.

T3. "Is 92 confidence 92% accurate?"

No. Agreement heuristic.

T4. "Does the extension read WhatsApp?"

Only if the user is on a web page and visible links are collected, or they paste text in the dashboard. It does not hook the WhatsApp app.

T5. "Prove you prevent all XSS/SQLi/SSRF."

We have specific controls; CORS and DNS-rebinding remain gaps. Never say "all."

T6. "Where is Flask?"

Not used.

T7. "Is community report ground truth?"

No. It is a 5% weight signal with caps.

T8. "What if the model file is missing?"

There is no model file to miss; fusion still runs.

18. Explain this to the jury

What is your project?

PhishShield AI (code name CyberGuard) is a multi-channel scam intelligence prototype: a FastAPI engine scores URLs and pasted messages using heuristics, optional threat-intel lookups, psychological keyword signals, sender checks, QR decode, and community reports; a React dashboard and a Manifest V3 extension display Simple and Technical views.

Why is it different?

It is not only a URL blacklist. It fuses message language and community votes, and it refuses to let fear-words alone become HIGH/CRITICAL.

Why AI?

We use "AI" as assisted decisioning: explainable fusion. We do **not** ship a trained neural phishing classifier in this repository.

Why PostgreSQL?

We **do not use it yet**. We use SQLite. Postgres is a planned production store.

Why React?

To ship a scanner, history, reports, and charts quickly as a SPA.

Why Python?

FastAPI, urllib, OpenCV, pytest — one language for API and detection logic.

How does phishing detection work?

Parse URL/text → heuristic URLChecker + intel → keyword psychology → brand/sender/community → weighted sum → persist → explain.

How does the risk score work?

Weighted average of ten signals, cap 100, psych-only cap 35, then banded into four levels.

What happens when your model is wrong?

There is no model to be wrong in the ML sense. Heuristics misfire: we show reasons, allow feedback, and keep psych from over-firing. Users should still think.

How do you prevent false positives?

Trusted domains, psych cap, explainability, human reports of "safe." Still imperfect.

How do you handle new phishing attacks?

New domains: age/DNS/TLD/intel may catch some; brand-new kits evade lists. v2 would add a trained model and faster intel.

How do you protect user data?

Password hashes, JWT, gitignored .env, extension doesn't scrape passwords. Scan URLs **are** stored. Default JWT secret is a known weakness.

How does the browser extension work?

MV3 worker + content script + popup; `fetch` to FastAPI; badge/overlay; visible links only.

How does frontend communicate with backend?

Axios JSON to `VITE_API_URL`.

How does backend communicate with the database?

`sqlite3.connect`, row factory, parameterized SQL.

What if the database goes down / file missing?

Startup `init_db()` creates tables. If the disk is broken, APIs that write fail with 500. Reads fail. UI shows connection/server errors.

What if ML fails?

Fusion is in-process Python. If a dependency throws, the route returns 500. Intel failures are fail-soft (None).

What makes this scalable?

Today: not really (SQLite, single local server, CORS *). Scaling story is Postgres + workers + restricted CORS + caching intel.

Biggest limitation?

No labelled ML, paste-only messengers, keyword evasion, public scan listing, local-only deploy.

Version 2?

PostgreSQL, packaged OCR, real ml signal, rate limits, HTTPS, Android companion, evaluation set, tighter authz on GET /scans.

19. Technology difficulty

Technology	Difficulty	Estimated learning time (for this project, not mastery)	Priority
Git basics	Easy	3–4 hours	HIGH
HTTP/JSON/REST	Easy	4–6 hours	HIGH
React components + hooks	Medium	2–3 days	HIGH
React Router + Axios auth	Medium	1 day	HIGH
Tailwind usage	Easy	4 hours	MEDIUM
Vite workflow	Easy	2 hours	MEDIUM
Python functions/modules	Easy–Medium	1–2 days	HIGH
FastAPI routers + Pydantic	Medium	1–2 days	HIGH
JWT + bcrypt concepts	Medium	4–6 hours	HIGH
SQLite + SQL	Medium	1–2 days	HIGH
URL heuristics (url_checker)	Medium	1 day reading code	HIGH
Fusion + risk bands	Medium	3–4 hours	HIGH
Psychology keywords	Easy	2 hours	HIGH

Technology	Difficulty	Estimated learning time (for this project, not mastery)	Priority
SSRF/SQLi/CORS concepts	Medium–Hard	1 day	HIGH
OpenCV QR	Medium	2–3 hours	MEDIUM
Manifest V3 messaging	Hard	1–2 days	HIGH
Threat intel APIs	Medium	3 hours	MEDIUM
pytest	Easy–Medium	3–4 hours	HIGH
Real sklearn/CNN (future)	Advanced	weeks	LOW now
PostgreSQL (future)	Medium	1–2 days	MEDIUM

20. Five-day crash course

Day 1 — Architecture + UI/API basics

Morning: read README, start uvicorn + Vite, hit `/health` and `/docs`. Draw the architecture in section 7 from memory.

Afternoon: walk `App.jsx` routes; log in; open Scanner. Read `api.js`.

Homework: explain CyberGuard vs PhishShield naming.

Day 2 — Backend + SQLite

Morning: `main.py` routers; `user.py` signup/login; `auth.py`.

Afternoon: `connection.py` every table; run a few SQL SELECTs on `threats.db` (copy first). Trace `POST /report`.

Homework: answer Q22–Q25 and Q71.

Day 3 — Detection + risk

Morning: `url_checker.py` lists (trusted, piracy, TLD, brands). `external_intel.py` fail-soft.

Afternoon: `psychology.py` + `risk_engine.py` with a calculator: plug fake signals, hit the psych cap. Read `content_analyzer.py` including `"m1": url_risk`.

Homework: 2-minute speech “we do not have a CNN.”

Day 4 — Extension + security + full trace

Morning: load unpacked extension; watch Network tab to :8000. Read manifest permissions.

Afternoon: `ssrf.py`, CORS line, report caps, screenshot 5 MB. Rehearse Path A and Path B in section 8.

Homework: trick questions T1–T8.

Day 5 — Viva

Morning: cheat sheet + section 18 answers out loud.

Afternoon: mock jury — one person only asks trick questions. No laptops for the first 20 minutes. Then open `/docs` and `risk_engine.py` only.

21. One-page cheat sheet (printable)

PROJECT: PhishShield AI (code: CyberGuard API 2.0)

PROBLEM: Scams arrive as links **and** as messages (SMS/WhatsApp/email). People need an explainable risk check, not a black box.

SOLUTION: FastAPI fusion engine + React dashboard + Chrome MV3 extension.

FRONTEND: React 19, Router 7, Axios, Tailwind 4, Vite 8, Recharts — `frontend/src/`

BACKEND: FastAPI + Uvicorn :8000 — `backend/app/`

DATABASE: SQLite `backend/threats.db` (PostgreSQL **planned**)

AI/ML: No trained model. `risk_engine.fuse` weighted signals. "m1" copies URL risk.

EXTENSION: Manifest V3 — `extension/` — `check-url` + `analyze/content`

KEY FEATURES: URL + paste + email fields + QR + optional screenshot OCR; Simple/Technical views; community reports; dashboard stats.

KEY NOVELTY: Multi-channel normalize → fuse; psych **cannot** solo HIGH/CRITICAL; confidence = agreement.

RISK SCORE: 0–100 weighted sum; LOW<30, MEDIUM<50, HIGH<70, else CRITICAL; psych-only cap 35.

IMPORTANT APIs: `POST /api/v1/analyze/content`, `POST /api/v1/threat/check-url`, `POST /api/v1/user/login`, `POST /api/v1/report`, `GET /api/v1/stats/overview`

IMPORTANT TABLES: `users`, `url_checks`, `threats`, `content_scans`, `extracted_links`, `user_reports`, `domain_reputation`, `scan_feedback`

TOP 10 VIVA QUESTIONS:

1. Why not Flask/Postgres/CNN? Because they are not in the running system.
2. How is the score calculated? Weighted fusion.
3. What is confidence? Signal agreement.
4. Can psychology be CRITICAL? No.
5. How do you stop SQL injection? Bound parameters.
6. How do you stop SSRF? Public-URL assertion.
7. Two scan pipelines? `check-url` vs `analyze/content`.
8. Does the extension read SMS? No.
9. What if intel is down? Fail soft, heuristics remain.
10. Biggest limitation? Prototype: no trained model, SQLite, open CORS, paste-only.

22. Code vs documentation discrepancies (memorize)

Claim you might hear	Code reality
Flask backend	FastAPI
PostgreSQL in production	SQLite <code>threats.db</code>
Trained CNN / sklearn	Absent; numpy for QR
URLhaus	OpenPhish feed in code
DATABASE_URL drives the DB	<code>connection.py</code> ignores it
JWT expires in 30 minutes	Routes issue 7-day tokens

Claim you might hear	Code reality
Global rate limit	Only report caps
Native SMS	Planned (<code>mobile_source.py</code>)
Screenshot always OCRs	Optional Tesseract
GET /scans is private	Public API
<code>model_probability</code> is ML	Equals fused score
SQLAlchemy is the data layer	Models unused by live routes
Product name CyberGuard only	SIH name PhishShield AI

End of guide. If the code changes, regenerate this document from the repository — do not edit the PDF by hand and drift.